Harborline Cruise Management Booking and Smart Operations System

(13 size) A Project Based Learning Report Submitted in partial fulfilment of the requirements for the award of the degree

of

Bachelor of Technology

in the Department of [Department Name]

[Course Name] ([Course Code])

Submitted by

Roll No.: [Roll Number] — [Student Name]
Roll No.: [Roll Number] — [Student Name]
Roll No.: [Roll Number] — [Student Name]

Under the guidance of

[Faculty Guide Name]



[Department Name]

Koneru Lakshmaiah Education Foundation, Aziz Nagar

Aziz Nagar – 500075

[Submission Month Year]

Abstract

Harborline is a role-based cruise management and booking prototype that connects passenger reservations with onboard and ship-operation workflows. Passengers can browse sample sailings, reserve a cabin, assign one dining seat to each ticketed guest across capacity-limited meal batches, and review itinerary notices. Crew can view shifts, record attendance, accept assigned faults or guest-service work, and log progress. Captains and managers can set up sailings, manage scoped staff permissions, control inventory, update port calls, maintain equipment, and lead muster workflows. A captain-reviewed disruption-recovery process links a delay or cancellation to affected booking parties, passenger choices, and claimable crew follow-ups. The application uses an HTML, CSS, and JavaScript interface, a same-origin Node.js HTTP API, and a local SQLite database. Server-side roles, sailing scopes, transactional booking, and activity logs support accountable workflows. Stripe Checkout can be enabled with server credentials; without them, bookings run in demo mode and take no payment. The project verification record dated 6 October 2026 reports 13 passing automated tests, but no external passenger or crew acceptance study, live vessel connection, or production deployment is recorded. This report explains the system, its development method, scenario-based evaluation, results, and implementation boundaries.

Keywords: cruise management; passenger booking; ship operations; Adaptive Software Engineering.

List of Figures

Figure 1. Harborline architecture and configured integration boundary 7

Figure 2. Role workspaces and key passenger and crew workflows..... 8

Figure 3. Captain-led dispatch and recovery control flow 9

List of Tables

Table 1. Implemented workflows by role.....9

Table 2. Prototype capabilities and integration boundaries 9

Table of Contents

1 Introduction 6

2 Methodology 6

a. Development method and adaptive cycle 6

b. System architecture and data model 7

c. Authentication and operational accountability 7

3 Experiments 8

a. Role-based operational scenarios 8

b. Verification method and evidence limits 8

4 Results 9

a. Delivered workflows and recorded outcomes 9

5 Conclusion and future work 10

References 10

Harborline Cruise Management Booking and Smart Operations System

1. Introduction

Cruise operations span a continuous journey from choosing a sailing to returning ashore. A reservation must represent the voyage, cabin, and the number of guests travelling together. Before embarkation, passengers need clear booking status and a usable boarding process. Once onboard, dining capacity, guest requests, port-call changes, maintenance findings, and safety drills involve different departments. If each activity is recorded in a separate or static tool, staff may repeat information or act on an out-of-date status, while passengers may see a change without a clear way to respond.

Harborline was developed as a local academic prototype to connect those guest and operator workflows in one role-aware system. The project evolved through iterative requirements: server-backed sign-in and saved bookings; dining batches with a seat assignment for every ticketed guest; controls over who can change inventory; and captain-created operational work that crew members can accept and log. A later operations requirement was to make a fault report actionable: it needs a sailing, location, priority, responsible department, owner, and recorded progress rather than a static example card.

The goal is to demonstrate an end-to-end management flow, not to reproduce a cruise line system of record. The passenger workspace supports sample-sailing discovery, cabin selection, reservations, dining allocation, itinerary notices, onboard plans, and service cases. The crew workspace exposes assigned shifts, attendance, work history, fault and service tasks, and selected maintenance actions. Captains and managers can configure sailings and port calls, apply sailing-scoped staff permissions, oversee equipment and muster activity, review disruption impacts, and inspect logged changes. Each view is backed by a local API and SQLite database rather than browser-only sample state [1], [2].

The system has four principal user roles. Passengers manage their own booking and guest allocations. Crew members see tasks within their permitted sailings and record acceptance, progress, and completion. Captains initiate or approve operational actions, including fault dispatch, itinerary updates, muster events, and disruption recovery. Managers configure inventory and sailing access, review operations, and handle administrative decisions. The interface and API enforce these roles separately; a user does not gain authority merely by navigating to a different page.

A particular design contribution is the captain-reviewed disruption workflow. For a delayed or cancelled port call, the application prepares an impact review for confirmed booking parties and linked shore plans. An authorized captain or manager reviews and approves the plan before passengers receive in-app notices. Passenger responses are recorded, and crew can claim follow-up tasks. The workflow links operational change, guest impact, and staff action in one audit trail. It is a distinctive capability in this prototype; no claim is made that it is new to the cruise industry.

The implementation boundary is important. Sample sailings, fares, cabins, and some operational records are seeded. The default booking flow is a demonstration and takes no money unless the server is configured for Stripe Checkout. Camera scanning depends on browser Code 39 support, with manual and keyboard-scanner alternatives. In-app notices do not send email or SMS. The local offline queue covers selected actions, but it is not guaranteed emergency communications. There is no live connection to cruise inventory, identity, crew scheduling, port-agent, maintenance-sensor, or safety systems. Harborline therefore serves as a reviewable software increment for further stakeholder validation rather than an operational vessel service [1], [4].

2. Methodology

a. Development Method and Adaptive Cycle

The project follows an Adaptive Software Engineering (ASE) loop supported by short Agile-style increments. Speculation established a working vision and recorded uncertainty about passenger inventory, onboard capacity, crew authority, safety requirements, external data ownership, and payment. Collaboration turned user-directed changes into role-specific workflows and acceptance criteria. Learning came from implementing those flows against a local server and reviewing the resulting behavior and recorded automated checks. Adaptation then changed the backlog and prototype, including guest-level dining assignments, sailing-scoped staff rights, captain-issued faults, crew work logs, and port-disruption recovery [3].

This is a development record, not evidence from a completed user study. The sample backlog contains illustrative scenarios, and no passenger or crew participant quotes or formal acceptance sessions are recorded. Product decisions in this report are therefore described as implemented requirements and local verification, not as statistically validated usability findings.

b. System Architecture and Data Model

The prototype uses a same-origin browser client built with HTML, CSS, and JavaScript, served by a small Node.js HTTP server. The server exposes authenticated JSON routes and stores users, sessions, sailings, cabins, bookings, dining batches and seats, shift and task records, permission scopes, port calls, safety events, maintenance schedules, service cases, and action logs in SQLite. This keeps the academic build simple to start while providing server-side persistence and transaction boundaries. Cabin reservations use collision protection and idempotency keys; dining assignments must match the booking party size, use available seats on the same sailing, and fit the capacity of each selected batch [1], [2].

Figure 1 shows the implemented local architecture alongside optional integrations. Stripe is an adapter that operates only when server-side keys and a signed webhook secret are configured. The development default uses demo mode. The client can queue selected actions locally while disconnected, then submit them for server verification after reconnection. The queue is limited and device-local; it does not replace a durable fleet-wide synchronization service.

c. Authentication and Operational Accountability

Passwords are stored as scrypt hashes, and authenticated sessions use random tokens stored by hash with HttpOnly and SameSite cookies. API handlers check role, account ownership, and sailing-level permission before exposing or changing records. Staff permissions distinguish actions such as managing a sailing, editing dining inventory, dispatching faults, viewing a crew job board, scanning boarding, and managing muster. Consequential operations append actor-attributed history, including inventory adjustments, fault acceptance and work, access changes, boarding scans, maintenance completion, port updates, and disruption decisions. These controls are implemented in the local prototype but have not received an independent security review or an operator audit-retention review [1], [2].

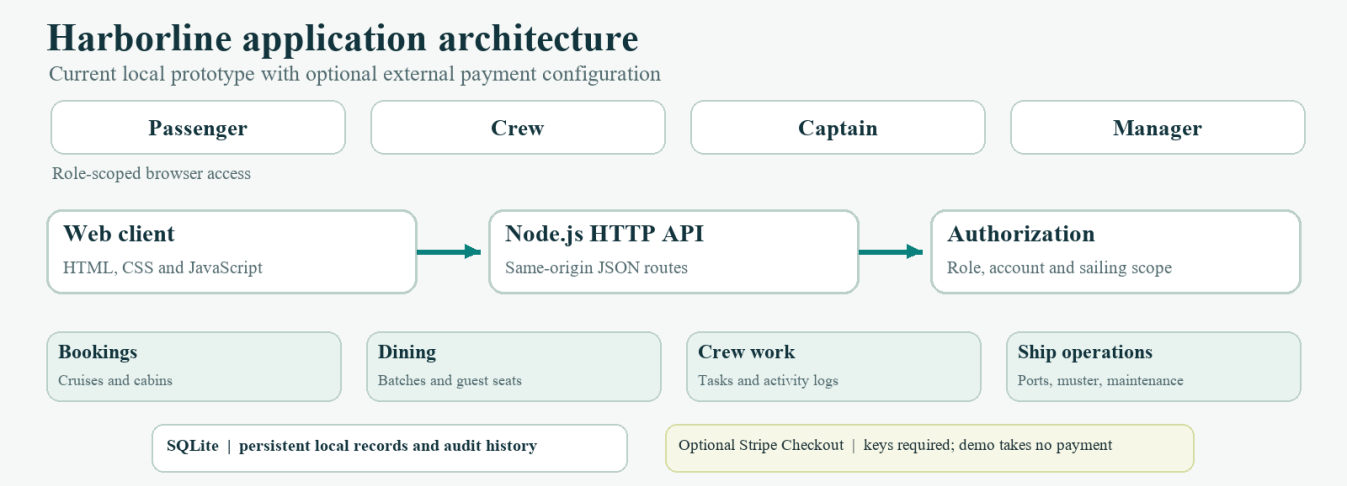


Figure 1. Harborline architecture and configured integration boundary

3. Experiments

a. Role-Based Operational Scenarios

Evaluation uses repeatable scenarios that exercise the principal roles and server-backed state. In the passenger scenario, a signed-in user selects a sample sailing, reserves an available cabin, and checks that a second request cannot claim an already reserved cabin. The passenger then assigns one dining seat to each guest in the booking. Each guest can be assigned to a different batch; blocked, filled, and available seats are shown, and the server rejects missing guest assignments, duplicate seats, blocked seats, or a batch over capacity.

In the crew and captain scenario, a captain or manager creates a fault with sailing, deck or location, department, priority, and task details. A crew member with access to that sailing can accept the work, start it, and record resolution. The action history identifies the reporter and crew activity. Staff-access settings constrain which sailings and actions each account can manage. A separate boarding scenario checks ticket references against the authorized sailing manifest, rejects repeat scans, and records the operator who checked in the booking.

For a port disruption, the captain or manager selects a delayed or cancelled call, reviews affected confirmed booking parties and linked shore plans, then approves the recovery. Only after approval are in-app notices created and passenger response options recorded. Claimable follow-up tasks are opened for crew and their progress is logged. Safety and preventive-maintenance scenarios verify that authorized staff can open a drill or emergency muster, account for ticketed guests at a configured station, and complete scheduled equipment service with a note and next due date. Figure 2 summarizes the user responsibilities covered by these scenarios.

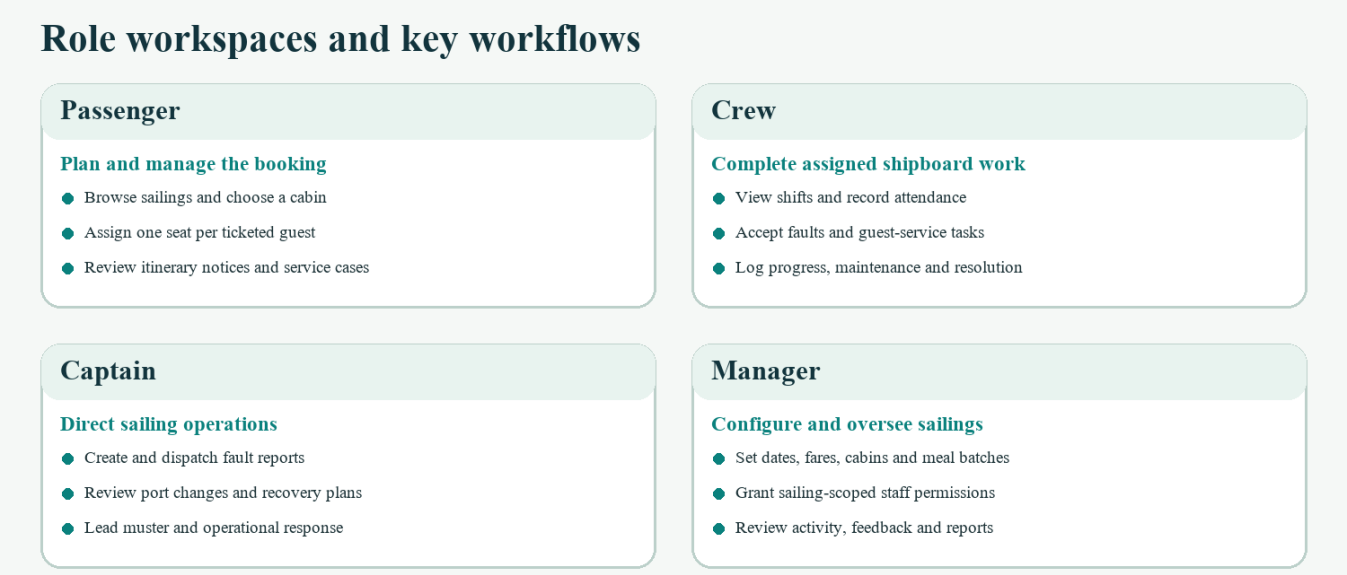


Figure 2. Responsibilities represented in the local passenger and operations prototype

b. Verification Method and Evidence Limits

The repository test plan defines passenger, crew, captain, and manager acceptance checks and records an existing verification run dated 6 October 2026: 13 automated tests passed, comprising 7 domain tests and 6 HTTP integration tests. The documented integration cases cover authentication and authorization, reservation collisions and idempotency, SQLite-backed workflows, feedback review, crew operations, and registration validation. Syntax checks and a browser walkthrough are also recorded in that project document [4].

These results are evidence for the prototype code and its seeded local workflows only. No performance or load benchmark, live payment transaction, camera-device compatibility matrix, external passenger or crew study, vessel-system integration, or production security assessment is recorded. This report does not treat scripted scenarios as stakeholder acceptance.

4. Results

a. Delivered Workflows and Recorded Outcomes

The local build demonstrates a connected passenger and operations flow. A passenger account can register and sign in, browse seeded cruise options, select a cabin, and keep a reservation in the server database. Transactional checks protect cabin inventory from conflicting reservation requests. The dining workflow models seat status by batch and keeps the number of guest assignments equal to the booking party size. Onboard requests, feedback, itinerary notices, and service cases are also attached to signed-in accounts.

Crew and ship operations are more than a static job list. Captains and managers can create and dispatch faults for sailings they control; crew can claim eligible jobs, update progress, and close work with a resolution note. Shift attendance, swaps, preventive maintenance, port-call status, staff permissions, boarding scans, and muster accountability have server routes and history records. The manager and captain views bring these operational records together. The disruption-recovery flow adds a human approval step before notices and follow-up tasks are created. Figure 3 illustrates this dispatch-to-closure control path.

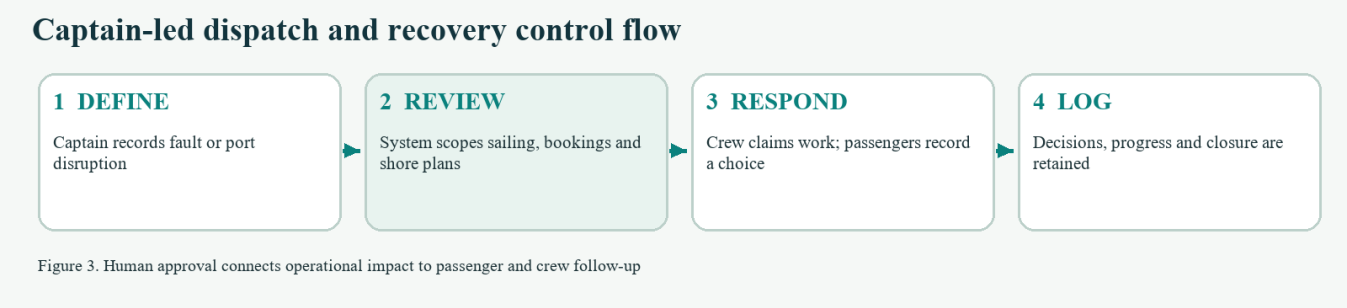


Figure 3. Human approval connects operational impact to passenger and crew follow-up

Table 1 Implemented workflows

Area	Status
Passenger	Cabin booking and guest-level batch seats
Crew	Accept, update and log assigned jobs
Captain	Dispatch faults and approve recovery
Manager	Configure inventory and staff access

Table 2 Integration boundaries

Area	Status
Payments	Stripe only when server keys are configured
Cruise data	Seeded sailings; no live operator feed
Boarding	Code 39/manual check-in; no identity adapter
Messages	In-app notices; no email or SMS delivery

Sample records and local infrastructure remain; deployment requires operator validation.

Tables 1 and 2 distinguish working local workflows from integration boundaries. The strongest result is traceability across roles: a configured permission controls who may act, the server persists the decision, and subsequent crew or passenger responses remain linked to the originating sailing or booking. The prototype still depends on seeded data and local infrastructure, so this result demonstrates coherent workflow behavior rather than fleet-wide operating effectiveness.

5. Conclusion and Future Work

Harborline demonstrates a role-based cruise-management prototype in which booking, dining allocation, boarding, guest service, crew work, maintenance, port status, and safety accountability share a persistent local backend. Its operational model gives captains and managers control over sailing-scoped actions while allowing crew to accept and log assigned jobs. Dining assignments are made per ticketed guest across capacity-limited batches. The captain-reviewed recovery workflow ties a port change to affected bookings, passenger choices, and crew follow-up, creating a concrete path from decision to resolution.

The work remains an academic local MVP. The next iteration should validate priorities with representative passengers, crew, captains, and managers; connect authoritative sailing and identity sources; complete production payment, refund, notification, and manifest flows; define schedule qualifications and labor rules; test offline recovery and device support; and establish backups, audit retention, accessibility review, deployment controls, and independent security assessment. Muster station mappings and emergency actions must be configured and validated by vessel operators before any operational use. Progress should be measured through task completion and error rates in consented usability sessions, alongside reliability and security checks, rather than through assumptions about user acceptance.

References

[1] Harborline project repository. README: implemented features, local setup, payment and scanner configuration, and production boundaries.

[2] Harborline project repository. server.js and app.js: Node.js API, SQLite persistence, authorization, booking, crew, and operations workflows.

[3] Harborline project repository. Architecture, Data Model and Risk Register; ASE and Agile Project Report; Backlog and Sprint Plan.

[4] Harborline project repository. Testing and Acceptance Plan, verification results dated 6 October 2026.